

Identifying causes of poor OpenMP parallel performance using Intel® VTune™ Amplifier

NAG Webinar

11th July 2018

Jonathan Boyle

HPC Application Analyst



Experts in numerical software and
High Performance Computing

My experience

- ▶ I'm a Research Software Engineer specialising in:
 - High Performance Computing
 - Numerical and scientific software
- ▶ My current role at the **Numerical Algorithms Group**
 - Analysing and optimizing software for NAG's clients
 - Serial & parallel code
 - Contributing code to the **NAG Numerical Library**
 - Contains 1700+ numerical and statistical routines
 - Recent work on EU funded "Performance Optimisation and Productivity" project
 - <https://pop-coe.eu/>

Caveat

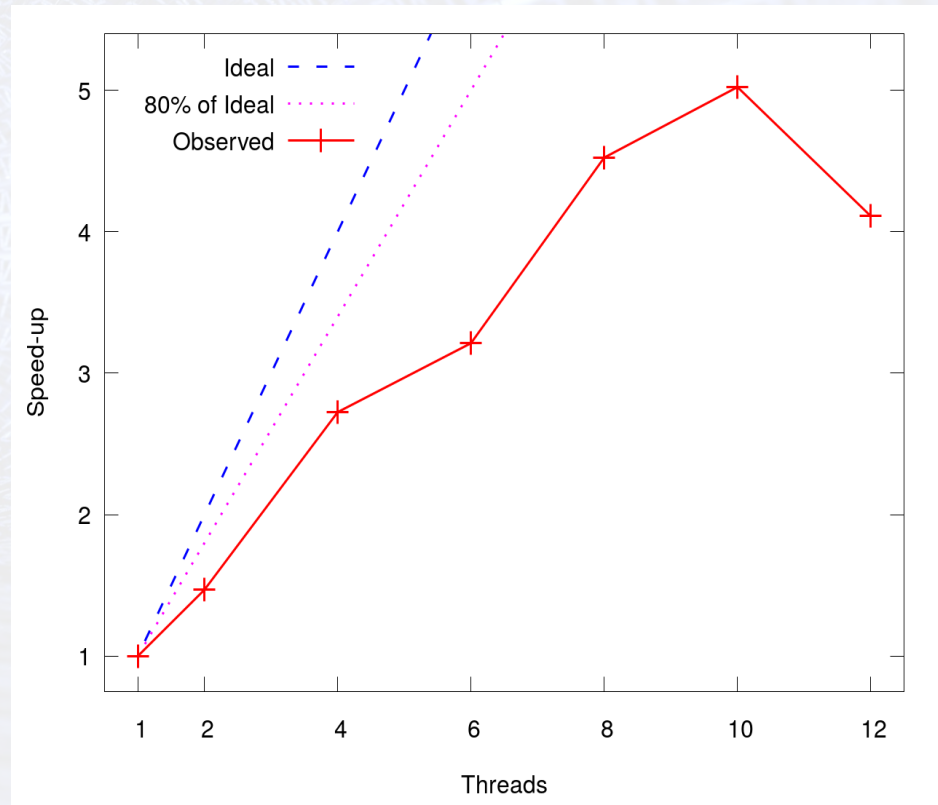
- ▶ This is not an exhaustive description
- ▶ The aim is to make it easy for you to start using VTune Amplifier to analyse OpenMP performance
- ▶ I recommend reading the VTune documentation
 - VTune contains a lot of additional functionality which won't be covered today
 - VTune functionality & support varies
 - e.g. OpenMP from nested parallelism is not supported
 - Online release notes describe **software & hardware** requirements
 - Plus **limitations**

Scope of talk

- ▶ Performance analysis of **OpenMP** parallelization
- ▶ **I assume you already optimized single thread performance, e.g.**
 - Compiler optimizations
 - Vectorization
 - Memory bandwidth bottlenecks eliminated
 - etc., etc.
- ▶ **Single thread optimization is important**
 - VTune is a good tool for optimizing serial execution
 - Usually used alongside Intel® Advisor

Motivation

- ▶ Performance analysis is usually necessary to get good performance from parallel code
- ▶ It's hard to interpret plots of speedup or global efficiency
 - They tell us if speedup is poor, but not the cause(s), and **there may be many causes**
- ▶ VTune Amplifier is a good tool for identifying causes of inefficiency in OpenMP parallelism



So we have a tool, we also need a methodology.....

- ▶ The idea: devise a useful set of performance metrics
 - e.g. one metric per source of inefficiency
- ▶ Then view trends in metrics
 - e.g. as a function of number of threads/cores
- ▶ Use metrics to guide focus of code optimization
- ▶ Metrics give a systematic way to compare performance
 - We can compare values before and after code changes
- ▶ The two main causes of poor parallel speedup are:
 1. Problems in the parallelisation
 2. Slow down in the computation

Top level OpenMP performance metrics

- ▶ **Efficiency Metrics** = Ideal Runtime / Actual Runtime
 - Ideal Runtime depends on the source(s) of inefficiency under consideration
- ▶ **Scaling Metrics** = Value for reference case (usually 1 thread) compared to value on n threads
- ▶ **Parallel Efficiency** i.e. how good is the parallelization?
 - Ideal Runtime = Average (over threads) of time in useful computation
 - Useful computation excludes time from adding parallelism to the execution
- ▶ **Computational Scaling** i.e. is the computation slowing?
 - Sum over threads of time in useful computation compared to 1 thread value
- ▶ **Global Efficiency**
 - Parallel Efficiency x Computational Scaling

Contributions to Parallel Inefficiency

▶ Amdahl's Law

- Inefficiency due to serial execution **outside** OpenMP regions

▶ Parallel Inefficiency within OpenMP regions

- This can be split into various contributions, e.g.
 - Load imbalance
 - Serialization (e.g. due to critical regions)
 - OpenMP Overheads

▶ For each of these we can define an **Ideal Runtime**

- And then calculate: $\text{Efficiency} = \text{Ideal Runtime} / \text{Runtime}$
- For OpenMP regions VTune reports 'Potential Gains'
 - $\text{Ideal Runtime} = \text{Runtime} - \text{Potential Gain}$
- VTune also reports time in serial execution outside parallel regions
 - It is then trivial to calculate 'Potential Gain' for this serial execution

Contributions to Computational Scaling

- ▶ Computational Scaling is a function of
 1. IPC (instructions per cycle)
 2. Instructions count
 3. Processor frequency
- ▶ Hence we can also define (for useful computation)
 - IPC scaling
 - Instruction count scaling
 - CPU Frequency scaling
- ▶ We can get these from VTune too
 - Note: VTune reports $CPI = 1 / IPC$

Using VTune

Before using VTune

- ▶ **Optional: use the APIs**
 - e.g. to switch VTune on/off to collect data for a region of interest
 - These are easy to use and well documented
- ▶ **Compile code with -g (Linux) or /DEBUG (Windows) flags**
 - I've used Intel compilers, but online info say it supports others (e.g. VS, GCC)
 - Optional: embed source filenames using `qparallel-source-info=2` or `/Qparallel-source-info=2`
- ▶ **Identify a suitable computation**
 - Something representative but not too long
 - You want to avoid collecting unmanageable amounts of data
 - And make sure you are using sensible thread placement!
- ▶ **Linux: set up the environment**
 - `source ampxe-vars.sh` (or `ampxe-vars.csh`)

Running VTune

- ▶ Choice of GUI or command line version
- ▶ Linux: start the GUI by running `amplxe-gui`
 - Or use command line version `amplxe-cl`
- ▶ Windows: start VTune with admin privileges
 - Required for accurate CPU time collection

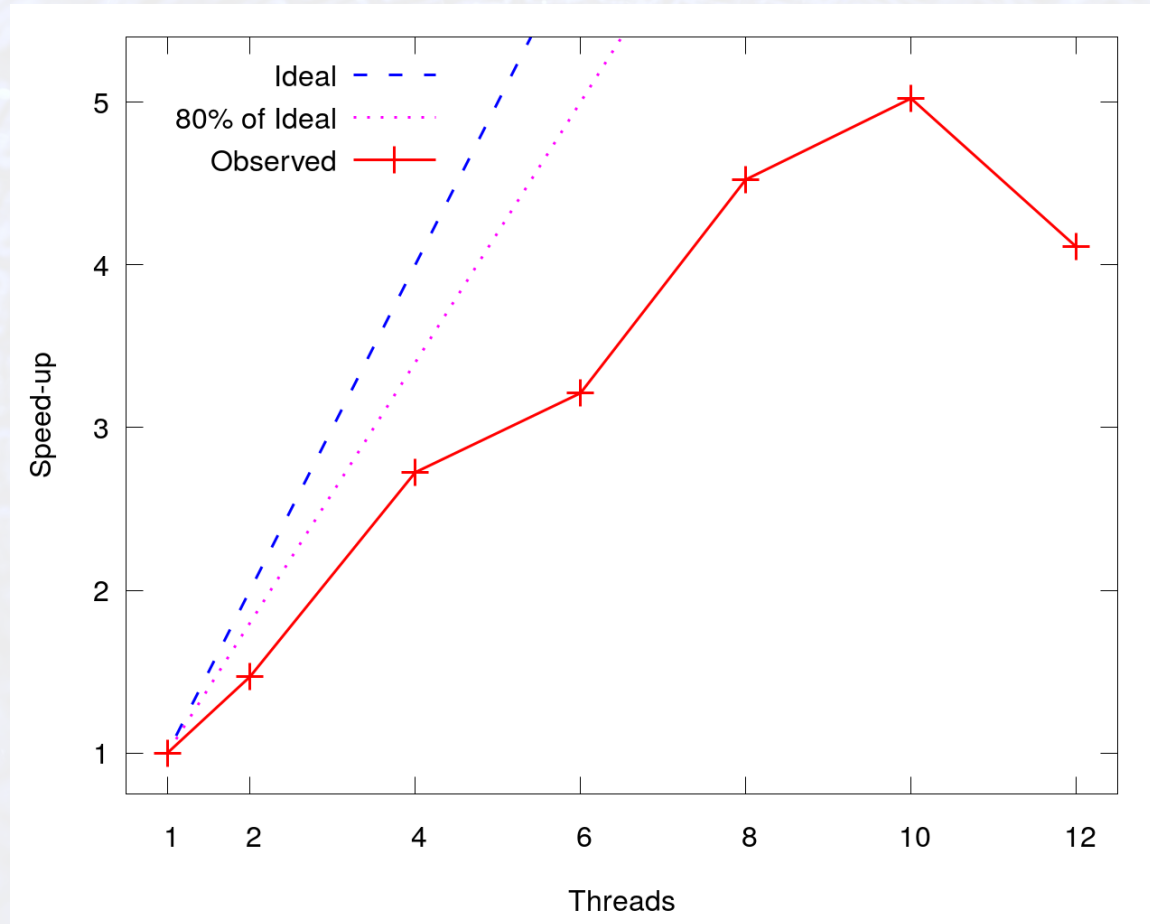
A warning

- ▶ **The amount of data can be overwhelming**
 - There are multiple 'viewpoints' & 'groupings'
 - Plus choice of Bottom-up, Caller/Callee, Top-down Tree views
 - Also a choice of various analysis types
 - **Is it not clear what is useful and what to ignore**
 - Also can be slow, e.g. several seconds to display a window
- ▶ **Finding the data you want can be difficult/slow if you don't know how to use it**

VTune GUI Demo

Metrics example

Initial speedup plot



Initial performance metrics

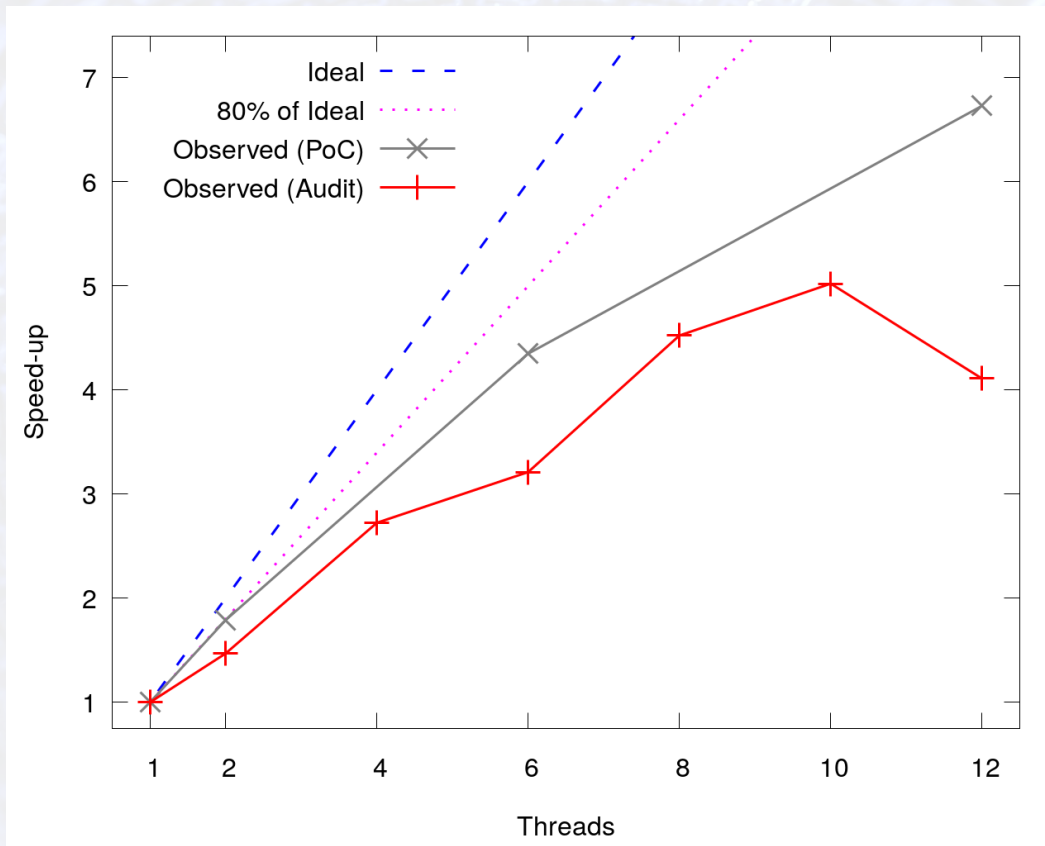
#Threads	1	6	12
Global Efficiency	0.97	0.52	0.33
Parallel Efficiency	0.97	0.64	0.50
Computational Scaling	1.00	0.82	0.66
IPC Scaling	1.00	0.92	0.91
Instructions Scaling	1.00	1.00	1.00
CPU Frequency Scaling	1.00	0.89	0.72
Amdahl's Law Efficiency	1.00	0.79	0.69
Load Balance Efficiency	1.00	0.85	0.85
OpenMP Overhead Efficiency	1.00	0.99	0.98

► What does this tell us?

- There is low Computational Scaling (caused by the Frequency Scaling)
- There is a large amount of serial execution outside OpenMP on 12 threads
- The load balance isn't ideal (this was located within one parallel loop)

► This was used to guide code optimization

Speedup for optimized code



- ▶ But did all the modifications improve performance?
- ▶ Can the code be improved further?

Example of OpenMP metrics

	Original code			Optimised code		
#Threads	1	6	12	1	6	12
Global Efficiency	0.97	0.52	0.33	1.00	0.73	0.56
Parallel Efficiency	0.97	0.64	0.50	1.00	0.89	0.76
Computational Scaling	1.00	0.82	0.66	1.00	0.82	0.74
IPC Scaling	1.00	0.92	0.91	1.00	0.88	0.83
Instructions Scaling	1.00	1.00	1.00	1.00	0.99	0.85
CPU Frequency Scaling	1.00	0.89	0.72	1.00	0.94	1.05
Amdahl's Law Efficiency	1.00	0.79	0.69	1.00	0.97	0.87
Load Balance Efficiency	1.00	0.85	0.85	1.00	0.89	0.76
OpenMP Overhead Efficiency	1.00	0.99	0.98	1.00	0.99	0.99

► Optimized code shows improved overall performance

- Improved Frequency Scaling & Amdahl's Law Efficiency
- There may still be room for improvement
 - Load balance efficiency for optimized code worse on 6 & 12 threads
 - IPC and Instructions Scaling are also worse

Summary

- ▶ Intel provide a nice tool for analysis of OpenMP performance
- ▶ A systematic approach using a suitable set of metrics is often essential for code optimization

Experts in High Performance Computing, Algorithms and Numerical Software Engineering

www.nag.com | blog.nag.com | @NAGtalk